

# Atlas / Argus Codebase Context Pack

Everything a future AI coding assistant needs to know before modifying the codebase

Generated from the uploaded backend ZIP, frontend ZIP, patch bundle, and blueprint PDFs on 2026-05-29

## Core philosophy

The projected public record is not the truth. The evidence chain is the truth. The projection is the current best explanation of that evidence.

## Contents

- 1. One-page operating brief
- 2. Source artifacts and validation status
- 3. Repository map and runtime topology
- 4. Backend architecture and source-of-truth model
- 5. Subsystem map
- 6. Data model, migrations, and repositories
- 7. API surface and security boundaries
- 8. Conflict, projection, provenance, and ingestion rules
- 9. Frontend app context
- 10. Product gaps and MVP build sequence
- 11. Development workflow and commands
- 12. AI coding rules and guardrails
- 13. Future enhancements and roadmap
- Appendices: file paths, glossary, and handoff checklist

# 1. One-page operating brief

## Read this before changing code

This is a mature FastAPI backend plus a real but unfinished React/Vite frontend scaffold. Do not rewrite the architecture. Extend the existing bounded contexts, preserve the claim/evidence/projection model, and ship changes as small vertical slices with tests.

| Topic               | What an AI must remember                                                                                                                                                                                                                                                                                                 |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Product identity    | Atlas / Argus is a claims-based aviation accident data integrity platform. It reconciles conflicting evidence into a defensible record for investigation, litigation, and safety workflows.                                                                                                                              |
| Source of truth     | Claims and evidence chains are the truth. Projected accident records are derived views. Editorial/public pages may explain evidence but must not silently replace structured evidence-backed facts.                                                                                                                      |
| Current backend     | Python 3.12, FastAPI, SQLAlchemy async,                                                                                                                                                                                                                                                                                  |
| Current frontend    | apps/web exists in atlas-web-app-docs-reports.zip: React + Vite + TypeScript + Tailwind + TanStack Query + React Router + Lucide, with login, cases, claims, conflicts, timeline, provenance, reports, audit, and document/report routes. It remains a scaffold/workbench that needs BFF, storage, and report hardening. |
| Missing MVP pieces  | Remaining MVP pieces: apps/marketing; wire and harden the services/bff skeleton; connect Documents/Reports routes to production BFF, storage, parsing, and immutable report export; shared EvidencePanel; conflict hardening validation; global search; full frontend/BFF tests.                                         |
| Security rule       | The browser must never hold X-API-Key. Human login belongs in the BFF. The BFF maps an HttpOnly session cookie to a server-side API key and proxies to FastAPI.                                                                                                                                                          |
| Tenant rule         | Public data may be visible to tenants. Tenant-private data must never leak publicly or across tenants. Tenant payload tables are protected by PostgreSQL RLS using app.current_tenant_id.                                                                                                                                |
| Primary workflow    | Ingest sources -> create claims -> detect conflicts -> resolve or reopen with rationale -> rebuild projection -> expose                                                                                                                                                                                                  |
| Highest-risk UI bug | Conflict-candidates status: applied in atlas-argus-docs-reports.zip. Preserve the candidates endpoint, stable claimId selection, selectedClaimId form behavior, 409 retry flow, and candidate tests; do not reintroduce index-based winning_claim_id selection.                                                          |

Default AI strategy: inspect first, extend existing modules, preserve use-case/UoW/repository boundaries, add migrations when persistence changes, update API schemas/tests, and keep public/private data boundaries explicit.

# 2. Source artifacts and validation status

## Artifacts reviewed

| Artifact | Role in this context pack |
|----------|---------------------------|
|----------|---------------------------|

|                                                       |                                                                                                                                                                                            |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| atlas-argus-production-ready(9).zip                   | Primary backend codebase. Contains src/atlas, tests, Alembic migrations 001-049, deployment files, ops runbooks, and CI configuration.                                                     |
| atlas-web-app(3).zip                                  | Existing authenticated SPA scaffold under apps/web.                                                                                                                                        |
| atlas-conflict-candidates-patch.zip                   | Preserve and harden rather than rebuilding from scratch. Targeted correctness patch for conflict claim candidate selection across backend and frontend. No migrations or new dependencies. |
| atlas-argus-blueprint-updated(2).pdf                  | Product/website/web-app blueprint. Its update pages supersede older no-frontend assumptions.                                                                                               |
| atlas-argus-implementation-plan-updated(2).pdf        | Build plan grounded in codebase reality. Also supersedes older no-frontend rows.                                                                                                           |
| Aviation-Safety-Atlas-Implementation-Blueprint(2).pdf | Concise architecture and phasing blueprint: BFF, conflict queue, EvidencePanel, document ingestion, report export.                                                                         |
| You are Claude Opus 4_7 acting...(2).txt              | Original staged backend evolution prompt: public pages, search, maps, causality, tenancy, FOQA/ASAP, NL search, metering, editorial, audit UX.                                             |

## Validation performed in this container

| Check                                | Result        | Notes                                                                                                                                                            |
|--------------------------------------|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| python -m compileall -q src tests    | PASS          | Python files parsed successfully from the extracted backend bundle.                                                                                              |
| alembic                              |               |                                                                                                                                                                  |
| ruff check .                         | Not run       | ruff was not installed in the container. Run in the project environment.                                                                                         |
| mypy src                             | Not run       | mypy was not installed in the container. Run in the project environment.                                                                                         |
| pytest --no-cov -m "not integration" | Not completed | Collection failed because runtime dependencies such as SQLAlchemy were not installed in this container. This is an environment issue, not proof of test failure. |
| Frontend npm build                   | Not run here  | Use npm ci && npm run build in apps/web. The uploaded implementation plan notes the same dependency/install limitation.                                          |

## 3. Repository map and runtime topology

### Backend layout

```

atlas-argus/
src/atlas/
  domain/      # Pure entities, enums, exceptions, domain services
  application/ # Use cases, DTO orchestration, application services, UoW interface
  infrastructure/ # SQLAlchemy ORM/repositories, sessions, event bus, ingestion adapters, observability
  presentation/ # FastAPI routers/schemas and typer CLI commands
tests/         # Unit/API/integration tests mirroring source tree
alembic/versions/ # Migrations 001_initial_schema.py through 049_fk_covering_indexes.py
deploy/free/    # Caddy + Postgres/PostGIS + Redis + PgBouncer + API + worker self-hosted stack
ops/           # Load tests, Prometheus alerts, runbooks
pyproject.toml  # Python package, lint/type/test configuration
README.md, ARCHITECTURE.md, TENANT_RLS.md, OPERATIONS.md

```

Frontend layout

```
apps/web/
src/app/      # Providers, router
src/components/  # AppLayout, CaseWorkspaceLayout, Sidebar, TopBar, StatusChip, Feedback
src/features/auth/  # Auth provider and API calls expected through BFF auth endpoints
src/features/cases/  # Case list/detail/evidence hooks
src/features/conflicts/  # Conflict hooks and 409 parsing
src/routes/      # Login, CaseList, CaseDashboard, Claims, Conflicts, Timeline, Provenance, Reports, Audit
src/lib/api.ts    # Single fetch wrapper, credentials: include, snake_case <-> camelCase conversion
src/types/api.ts  # Frontend DTOs mirroring backend/BFF responses
```

Expected future monorepo additions

```
apps/marketing/      # Static public site: Home, Platform, Use Cases, Security, Compliance, Request Access
services/bff/        # Browser-safe session/API-key bridge; owns human users/sessions/CSRF
packages/ui/         # Optional shared UI primitives later
packages/types/      # Optional generated or shared DTOs later
```

4. Backend architecture and source-of-truth model

Clean architecture rule

| Layer          | Owns                                                                                                                       | Should not do                                                                                     |
|----------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| domain         | Entities, enums, pure invariants, deterministic services such as winner policy, conflict detection, normalization helpers. | Import FastAPI, SQLAlchemy sessions, Pydantic API schemas, or commit transactions.                |
| application    | Use cases and orchestration. Owns unit-of-work transaction boundary and calls commit once at the end.                      | Contain framework route code or direct SQLAlchemy model manipulation unless through repositories. |
| infrastructure | SQLAlchemy ORM models, repositories, DB sessions, workers, concrete adapters.                                              | Choose business policy that belongs in domain/use cases.                                          |
| presentation   | FastAPI routers, Pydantic request/response schemas, auth dependencies, Typer CLI.                                          | Implement core business rules that should be reusable outside HTTP.                               |

Evidence-chain model

```
Source -> IngestionRun -> RawSnapshot
      -> Claim(field_name, field_value, source_id, event_id)
      -> ClaimConflict(when active claims disagree)
      -> ProjectedAccidentRecord(derived current explanation)
      -> Provenance, Audit, Timeline, Report views
```

- Ingestion writes claims and claim history; projection is rebuilt from active claims.
- A conflict is not a bug. It is a first-class state that prevents silent false certainty.
- Manual override is represented as a MANUAL\_OVERRIDE claim under the CuratorOverride source, not as an edit to the projection table.
- Public pages store publication/editorial metadata and narrative. They should reference projected records and evidence, not duplicate structured truth.

Core lifecycle vocabulary

| Concept   | Values / behavior                                                                     |
|-----------|---------------------------------------------------------------------------------------|
| ClaimType | RAW, CONFIRMED, MANUAL_OVERRIDE, SUPERSEDED.<br>Active values are RAW, CONFIRMED, and |

|                |                                                                                                                                                                                                   |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ConflictStatus | MANUAL_OVERRIDE. SUPERSEDED claims remain auditable but do not win projections. OPEN and RESOLVED. OPEN fields project as DISPUTED. RESOLVED fields project to the winning claim/manual override. |
| Roles          | analyst = read-only; reviewer = analyst + ingest + resolve/reopen + review actions; admin = reviewer + merge/rebuild/admin/outbox/metrics.                                                        |
| RequiredField  | event_date, location, aircraft_type, fatalities_total, injuries_total, operator, registration, flight_phase, narrative.                                                                           |
| OutboxStatus   | PENDING, PROCESSING, PROCESSED, FAILED, DEAD_LETTER with exponential backoff and stale-lock recovery.                                                                                             |

## 5. Subsystem map

| Subsystem             | Purpose                                                                        | Primary paths / APIs                                                                                                                                                                            | AI guidance                                                                          |
|-----------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Atlas core            | Claims, conflicts, projections, provenance, ingestion replay/idempotency.      | domain/entities.py, services/ingestion.py, application/use_cases/ingest_source_data.py, resolve_conflict.py, reopen_conflict.py, reproject_event.py; /api/v1/accidents, /conflicts, /ingestion. | Do not bypass claims to mutate projected facts.                                      |
| Argus                 | Signal/anomaly/pattern detection across events.                                | domain/services/argus_*.py, use_cases/run_argus_signal_detection.py, routers/argus.py; /api/v1/argus/*.                                                                                         | Treat signals as reviewable observations, not truth.                                 |
| Chronos               | Timeline extraction and sequencing.                                            | domain/services/chronos_*.py, use_cases/extract_chronos_timeline_from_event.py, routers/chronos.py; /api/v1/chronos/* and public timeline.                                                      | Timelines must link back to evidence claims/sources.                                 |
| Orion                 | Entity and relationship extraction: aircraft, operator, airport, country, etc. | domain/services/orion_*.py, repositories/orion.py, routers/orion.py; /api/v1/orion/*.                                                                                                           | Extend Orion before creating a separate graph model unless causality truly needs it. |
| Hermes                | Source registry, crawl targets, fetch jobs, fetched documents, source changes. | routers/hermes.py, use_cases/register_hermes_source.py, enqueue/run fetch job, infrastructure/ingestion.                                                                                        | Server-side fetches are SSRF-sensitive; use allowlists and URL safety logic.         |
| Publication/editorial | Public event pages, revisions, publication workflow.                           | domain/publication, use_cases/public_events.py, editorial.py, routers/public.py, editorial.py.                                                                                                  | Pages are overlays/metadata/narrative; structured facts stay in projections/claims.  |

|                    |                                                                                   |                                                                                |                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Search/NL search   | Postgres FTS search and deterministic natural-language parsing.                   | domain/search, nl_search; routers/search.py, nl_search.py.                     | Public search indexes only public projections/page metadata; no private evidence by default.                        |
| Maps               | PostGIS map entries, event points, clusters.                                      | domain/maps, repositories/maps.py, routers/maps.py; /api/v1/maps/*.            | Always bound/limit map queries. No unbounded public exports.                                                        |
| Tenancy/enterprise | Tenant, membership, private sources/claims/overlays/safety reports/crossref.      | domain/tenancy, use_cases/tenancy.py, tenant_ingestion.py, routers/tenancy.py. | Enforce tenant boundaries in repos/use cases plus RLS. Never leak into public search/maps/pages.                    |
| Causality          | HFACS/SHELO attribution and interactions.                                         | domain/causality, routers/causality.py.                                        | Every causal assertion must be evidence-backed and reviewable. Do not infer causation from correlation or LLM text. |
| CMS                | Glossary, methodology, changelog public/editorial content.                        | domain/cms, routers/cms.py.                                                    | General content only; not a place to edit accident facts.                                                           |
| Audit/metering     | Audit views and usage events/rollups.                                             | routers/audit.py, metering.py; domain/metering.                                | Billing/metering must remain separate from core domain logic.                                                       |
| Echo crossref      | Cross-reference/precedent matching between tenant/private data and public corpus. | domain/crossref, application/crossref, CROSSREF_ENGINE.md.                     | Crosses public/private boundary by design; keep boundary auditable and tenant-scoped.                               |

## 6. Data model, migrations, and repositories

### Important ORM model groups

| Group                       | Models                                                                                                                                                                                                                                    |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Core evidence               | SourceModel, IngestionRunModel, RawSnapshotModel, AccidentEventModel, ClaimModel, ClaimHistoryModel, ClaimConflictModel, ClaimConflictClaimModel, ConflictActivityLogModel, ProjectedAccidentRecordModel, AccidentProjectionHistoryModel. |
| Outbox/archive/auth         | OutboxEventModel, OutboxWorkerHeartbeatModel, ArchiveManifestModel, ApiKeyModel, PendingDuplicateReviewModel, EventIdentityIndexModel.                                                                                                    |
| Orion/Chronos/Hermes/Argus  | OrionEntityModel, OrionRelationshipModel, ChronosTimelineEventModel, HermesSourceModel, HermesFetchJobModel, HermesFetchedDocumentModel, ArgusSignalModel, ArgusSignalEvidenceModel, ArgusSignalReviewModel.                              |
| Publication/search/maps/CMS | PublicEventPageModel, PublicEventPageRevisionModel, SearchIndexEntryModel, MapIndexEntryModel, GlossaryTermModel, MethodologyPageModel, ChangelogEntryModel and revisions.                                                                |
| Tenancy/enterprise          | TenantModel, TenantMembershipModel, TenantSourceModel, TenantIngestionRunModel, TenantClaimModel, TenantEventOverlayModel,                                                                                                                |

Causality/NL/metering

TenantSafetyReportModel, TenantEventAssociationModel,  
TenantCrossrefResultModel.  
HfacsCategoryModel, HfacsSubcategoryModel,  
EventHfacsAttributionModel, SheloFactorModel,  
SheloFactorInteractionModel, NlQueryLogModel,  
SavedNlQueryModel, UsageEventModel,  
UsageDailyRollupModel.

## Repository and UoW rules

- All concrete repositories are attached to SQLAlchemyUnitOfWork in infrastructure/db/unit\_of\_work.py.
- Repositories query and mutate but do not commit. Use cases own transaction boundaries and call await uow.commit().
- create\_uow() yields the normal system/public UoW. create\_tenant\_uow(tenant\_id) establishes the transaction-local tenant RLS context.
- SQLAlchemyTenantUnitOfWork reapplies app.current\_tenant\_id after commit/rollback because PostgreSQL GUC is transaction-local.
- When adding persistence, update ORM model, migration, repository interface/implementation, use case tests, and API schemas as needed.

## Alembic status

- Migrations are in alembic/versions and currently reach 049\_fk\_covering\_indexes.py.
- Major milestones include initial schema, security hardening, ingestion identity, lifecycle constraints, Orion, Chronos, Hermes, Argus, public pages, editorial workflow, search, tenancy/overlays, maps, CMS, tenant ingestion, causality, NL search, metering, tenant RLS, crossref results, durable outbox, and FK covering indexes.
- Use alembic upgrade head to apply. Use ATLAS\_ALLOW\_DB\_RESET=1 make migrate-check only against disposable databases.

# 7. API surface and security boundaries

## Router families

| Router                                 | Representative paths                                                                                                                     |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| accidents/provenance                   | GET /api/v1/accidents/{event_id}; POST /api/v1/accidents/{event_id}/reproject; GET /api/v1/accidents/{event_id}/provenance.              |
| conflicts                              | GET /api/v1/conflicts; GET /api/v1/conflicts/{conflict_id}; GET /history; POST /resolve; POST /reopen. Patch adds GET /candidates.       |
| ingestion                              | POST /api/v1/ingestion/sources/{source_id}.                                                                                              |
| public events                          | GET /api/v1/public/events; GET /{slug}; GET /evidence; GET /timeline; GET /related; GET /audit.                                          |
| editorial/CMS                          | /api/v1/editorial/pages/* plus /api/v1/public/glossary, methodology, changelog and /api/v1/editorial/* CMS endpoints.                    |
| search/maps/NL                         | /api/v1/search/events; /api/v1/admin/search/reindex; /api/v1/maps/events; /api/v1/maps/events/cluster; /api/v1/search/nl.                |
| Orion/Chronos/Hermes/Argus             | /api/v1/orion/*; /api/v1/chronos/*; /api/v1/hermes/*; /api/v1/argus*.                                                                    |
| tenancy/metering/causality/audit/admin | /api/v1/enterprise/tenants/*; /api/v1/admin/usage/*; /api/v1/public/hfacs/*; /api/v1/editorial/hfacs/*; /api/v1/audit/*; /api/v1/admin*. |



Authentication and authorization

- Every non-health endpoint requires X-API-Key in the FastAPI core, including routes named public. Public means published/read-only Atlas record, not anonymous internet access.
- API keys are stored HMAC-SHA256 when API\_KEY\_HASH\_SECRET is configured, with API\_KEY\_HASH\_SECRET\_PREVIOUS for staged rotation.
- Validated keys are cached briefly in-process. Set API\_KEY\_CACHE\_TTL\_SECONDS=0 for high-assurance immediate revocation.
- Role enum: analyst, reviewer, admin. Do not invent viewer/curator/tenant-admin labels unless mapped to the existing enum.
- The BFF must be the only browser-facing holder of raw upstream API keys. Use HttpOnly Secure SameSite=Lax cookies and CSRF protection for mutations.

Tenant isolation

- Tenant payload tables use PostgreSQL RLS with FORCE ROW LEVEL SECURITY and policy tenant\_id::text = current\_setting(app.current\_tenant\_id, true).
- The tenant runtime DB role must be NOSUPERUSER NOBYPASSRLS. A separate system role may use BYPASSRLS for explicit cross-tenant jobs.
- Unset tenant context fails closed: tenant payload reads return zero rows, not all rows.
- tenants and tenant\_memberships are intentionally not behind the same RLS policy because they are needed to establish context.

8. Conflict, projection, provenance, and ingestion rules

Ingestion rules

1. Validate JSON shape, raw payload size, and fingerprint before mutable state is consulted.
2. Use idempotency key semantics: same key and same full submission replays; same key and different payload rejects.
3. Resolve source and event identity before writing claims. Identity-critical normalization errors reject ingestion; non-critical errors preserve raw values under canonical field names.
4. Write one Claim per field and a ClaimHistory row with action=created.
5. Detect conflicts for fields where active claims disagree. Reopen resolved conflicts when new evidence contradicts the winner, but preserve the asymmetry described below.
6. Write CLAIMS\_UPDATED outbox event and finish the ingestion run.

Conflict resolve / reopen rules

| Action                    | Rules                                                                                                                                                                                                                                                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Resolve                   | Requires expected_version optimistic lock. Exactly one of winning_claim_id or manual_override_value. Manual override creates a MANUAL_OVERRIDE claim under CuratorOverride. Other active claims for the field become SUPERSEDED. Conflict status becomes RESOLVED and projection rebuilds immediately. |
| User reopen               | Requires expected_version. Reactivates claims superseded by the previous winner using ClaimHistory, clears winning_claim_id, status becomes OPEN, projection returns DISPUTED.                                                                                                                         |
| Evidence-triggered reopen | New contradictory source evidence can reopen a resolved conflict without reactivating older losers. The prior resolution stood at the time; the new dispute is between the prior winner and new claim.                                                                                                 |
| 409 behavior              | ConflictModifiedError is expected for stale expected_version. UI must display refreshed conflict/currentVersion/latestActivity/accidentRecord and force the reviewer to re-evaluate.                                                                                                                   |

Projection builder and winner policy

- Projection is rebuilt from active claims for the event. It is idempotent for a caused\_by\_outbox\_event\_id.
- Open conflict rows are authoritative: projected field becomes DISPUTED.
- Resolved conflicts with valid active winners project the winning value.
- Defensive value-set checks prevent false uncontested projections if conflict detection lags.
- Authority tiers: MANUAL\_OVERRIDE > CONFIRMED > RAW. Within equal type, lower reliability\_tier is more trusted. Ties use older created\_at and deterministic id ordering.
- completeness\_score is based on non-null fields over the required field count.

Provenance and audit

- Provenance queries canonicalize merged/absorbed events by default but expose absorbed\_event\_id so clients know redirection occurred.
- High-cardinality provenance collections are keyset-paginated with per-stream cursors.
- include\_archive=True currently raises NotImplementedError for provenance/conflict history archive retrieval; archive\_available may be false.
- Reports and audit UX should expose evidence chain, curator identity/rationale, timestamps, reliability tier, and hashes rather than only projected values.

9. Frontend app context

Current apps/web reality

| Area              | Current state                                                                                                                                                                                                   |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stack             | React 19, Vite 8, TypeScript 6, Tailwind 3, TanStack Query 5, React Router 7, Lucide.                                                                                                                           |
| Routes            | /login, /app, /app/cases, /app/cases/:slug, claims, conflicts, timeline, provenance, reports, /app/audit.                                                                                                       |
| API client        | src/lib/api.ts is the only server fetch module. It uses VITE_API_BASE_URL, credentials: include, JSON Content-Type, snake_case/camelCase conversion, and typed ApiError with 409 helper.                        |
| Layout            | AppLayout plus CaseWorkspaceLayout with left rail, center detail/action, right evidence panel. This is the correct spine for claims, conflicts, timeline, provenance, reports.                                  |
| Auth expectation  | Frontend calls /api/v1/auth/login, /api/v1/auth/me, /api/v1/auth/logout, but FastAPI core currently has API-key auth, not human sessions. The BFF must supply these routes or the callers must be standardized. |
| Reports/Documents | Routes/Documents.tsx and routes/Reports.tsx now exist in atlas-web-app-docs-reports.zip; finish BFF, storage, parse-status, immutable export, and download wiring.                                              |

Conflict candidates patch

Conflict candidates patch status

The conflict-candidates patch is treated as applied in atlas-argus-docs-reports.zip. Keep the authenticated /api/v1/conflicts/{conflict\_id}/candidates endpoint and selectedClaimId UI path; verify the patched backend/web trees before release.

| Patch part     | Change                                                                                 |
|----------------|----------------------------------------------------------------------------------------|
| Backend schema | Adds ConflictClaimCandidateDTO and ConflictCandidatesResponse in schemas/conflicts.py. |

|                |                                                                                                                                                  |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend route  | Adds GET /api/v1/conflicts/{conflict_id}/candidates with reader roles. Composes conflict, claims, and source metadata.                           |
| Frontend hooks | Adds useConflictCandidates(conflictId) and cache invalidation after resolve/reopen.                                                              |
| Frontend form  | Uses selectedClaimId instead of selectedClaimIdx, renders active candidates, disables superseded candidates, and removes unsafe raw ID fallback. |
| Tests          | Adds tests/api/test_conflicts_candidates.py with auth, 404, sorting, winning, superseded, missing source, empty response, and role coverage.     |

## Frontend architectural target

apps/web      # authenticated SPA, talks only to BFF  
 services/bff    # human auth, sessions, CSRF, upstream X-API-Key proxy  
 apps/marketing   # public static website, request-access and contact forms

- Preserve apps/web. Do not rebuild from scratch.
- Normalize BFF prefixes. The app currently expects /api/v1/auth/\*; either preserve that at the BFF or update all callers consistently.
- Build one shared EvidencePanel used by claims, conflicts, timeline, provenance, and reports.
- Make 409 conflict-modified UX an inline reviewed state, not a generic toast.
- Add Playwright coverage for login, case load, conflict resolve/reopen, 409 retry, reports, and upload states.

## 10. Product gaps and MVP build sequence

### Critical MVP gaps

| Gap                           | Why it matters                                                                                                      | Notes for implementation                                                                    |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| BFF/session layer             | A skeleton now exists in atlas-bff-docs-reports.zip, but production DB/session/CSRF/API-key wiring is not complete. | Wire and harden the existing service; do not create a second BFF scaffold.                  |
| Conflict queue hardening      | Still the killer workflow; candidate-ID correctness must stay stable.                                               | Patch is applied in atlas-argus-docs-reports.zip; preserve refreshed 409 panel flow.        |
| Unified EvidencePanel         | Every fact must be traceable without breaking workflow.                                                             | Use the same component across claims, conflicts, timeline, provenance, and report preview.  |
| Document upload/ingestion     | Routes/Documents.tsx now exists; users still need full PDF/docket intake wired to backend storage/parsing.          | Finish BFF/storage/parser/status integration; preserve raw bytes and extracted text views.  |
| Report export                 | The sellable deliverable is a court/submission-ready PDF with footnoted evidence chain.                             | Read projection/provenance/history; generate immutable versioned reports with content hash. |
| Marketing site/request access | Needed for qualified leads and trust review.                                                                        | Separate apps/marketing; no app auth dependency; sober copy, security/compliance pages.     |

### Recommended build order

7. Wire and harden the existing services/bff skeleton: login, me, logout, sessions, CSRF, server-side API-key bridge.
8. Normalize API route prefixes and ensure apps/web only talks to the BFF origin.
9. Verify the conflict-candidates patch remains applied; preserve candidate endpoint, schema, and selectedClaimId tests.

10. Harden app shell: protected routes, role-aware sidebar, case list, case dashboard.
11. Build shared EvidencePanel and wire it to claims, conflicts, timeline, provenance.
12. Complete conflict queue: stable candidate IDs, resolve/reopen, 409 refresh, history panel.
13. Add PDF/docket upload and ingestion status UI plus backend storage/extraction adapter.
14. Add report preview/generate/list/download and immutable report storage.
15. Build apps/marketing and request-access/contact flows.
16. Add frontend unit/component tests, BFF tests, and Playwright end-to-end flows.

## 11. Development workflow and commands

### Backend quick start

```
python -m venv .venv && source .venv/bin/activate
pip install -e '[dev]'
cp .env.example .env
alembic upgrade head
atlas bootstrap
uvicorn atlas.presentation.api.app:app --reload
```

### Backend checks

```
python -m compileall -q src tests alembic
ruff check .
mypy src
pytest --no-cov -m "not integration"
make test-unit
make test-integration # requires Postgres and migrations
make check
```

### Operational commands

```
atlas bootstrap --role admin
atlas outbox-process --limit 100
atlas outbox-worker --sleep-seconds 5
atlas hermes-worker --sleep-seconds 5 --batch-limit 5
atlas projections-rebuild --event-id <uuid>
atlas projections-rebuild --all
```

### Frontend commands

```
cd apps/web
npm ci
npm run build
npm run lint
npm run dev
```

### Production boot sequence summary

17. Validate environment and secrets. In deploy/free, run ./check-env.sh .env.
18. Apply schema with alembic upgrade head before starting the API.
19. Seed first admin key with atlas bootstrap when needed.
20. Confirm /health and /ready.
21. Run authenticated smoke request and verify /metrics access controls.

## 12. AI coding rules and guardrails

### Non-negotiable rules

- Do not rewrite the backend or flatten clean architecture. The package root is src/atlas.
- Do not create a second truth table for accident facts. Use claims, conflict resolution, manual override claims, and projections.

- Do not place human login/password/session into the FastAPI core unless the product decision explicitly changes. The intended place is services/bff.
- Do not put raw X-API-Key in browser code, localStorage, sessionStorage, or frontend environment variables.
- Do not expose tenant-private data through public pages, public search, public maps, reports, or NL answers.
- Do not create public causal claims without evidence and review. Treat causal correlations and LLM summaries as unreviewed until persisted through causality workflow.
- Do not use offset pagination for high-volume public lists, provenance streams, conflicts history, search, or map results where cursor/keyset pagination is appropriate.
- Do not write repositories that commit. Use cases commit. The UoW dependency rolls back on unhandled exceptions.
- Do not build empty architecture. Add bounded contexts only when a concept is truly separate.

Preferred implementation pattern

- 22. Inspect existing subsystem ownership first.
- 23. Add or update domain entity/enum/service only when the invariant belongs in domain.
- 24. Add a use case for orchestration and transaction boundary.
- 25. Add repository interface/SQL implementation when persistence access changes.
- 26. Add Alembic migration for DB changes and test migration/ORM consistency.
- 27. Add Pydantic schemas and FastAPI route as thin adapters.
- 28. Add unit tests for domain/use case behavior and API tests for route contracts/roles/errors.
- 29. Run compileall, ruff, mypy, and unit tests; document any unavailable tool/environment issue.

Prompting future AIs

Recommended opening instruction

Before writing code, inspect the relevant existing subsystem. Preserve naming, dependency injection, UnitOfWork boundaries, repository style, and tests. If you cannot run a command, say exactly why and continue with static review.

| Feature request              | Where to look first                                                                              |
|------------------------------|--------------------------------------------------------------------------------------------------|
| Public accident pages        | domain/publication, use_cases/public_events.py, routers/public.py, projected records/provenance. |
| Search                       | domain/search, use_cases/reindex_public_events.py, search_events.py, repositories/search.py.     |
| Maps                         | domain/maps, repositories/maps.py, routers/maps.py, migration 039_map_index.py.                  |
| Timelines                    | Chronos domain/services/use cases and public timeline endpoint.                                  |
| Entities/relationships       | Orion domain and repositories before creating any new graph system.                              |
| Signals/duplicates/anomalies | Argus and duplicate review use cases.                                                            |
| Causality                    | domain/causality, HFACS/SHELO models, causality router.                                          |
| Tenant/private overlays      | domain/tenancy, create_tenant_uow, RLS docs, tenant_ingestion use cases.                         |
| Billing                      | metering domain/use cases/routers; keep out of core claim logic.                                 |
| Frontend auth                | services/bff design; apps/web auth provider and api client.                                      |

13. Future enhancements and roadmap

| Area                | Future direction                                                                                               | Cautions                                                                     |
|---------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Public encyclopedia | Stable slugs, publication workflow, evidence-visible public event detail, related events, timeline, citations. | Public pages are projections plus editorial metadata, not independent truth. |
| Rich event search   | Structured filters: keyword, aircraft, operator, date range, location,                                         | Index public projections/page metadata by default; no private/raw            |

|                     |                                                                                                                          |                                                                                                     |
|---------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
|                     | airport, phase, fatalities, investigation status, confidence, source type.                                               | evidence.                                                                                           |
| Geospatial map UI   | Bounding box queries, clustering, hotspots, filters, MapLibre/Deck/Cesium compatible DTOs.                               | Enforce bounds and limits; protect PostGIS from expensive unbounded scans.                          |
| Causality graph     | Relational graph-compatible HFACS/SHELO assertions with provenance, confidence, review status, disputed interpretations. | No Neo4j dependency until relational model proves insufficient.                                     |
| Enterprise overlays | Tenant-private sources, claims, projections, FOQA/ASAP foundations, private overlays on public events.                   | RLS plus application checks; public data can flow into tenant context, tenant data cannot flow out. |
| NL search/answers   | Deterministic parser first, later embeddings/LLM summarization with grounding verification.                              | Natural-language answers must cite persisted evidence and never invent causal claims.               |
| Reports             | Immutable PDFs with source footnotes, hashes, export history, authorized downloads.                                      | Do not regenerate old versions in place. Version and hash them.                                     |
| Audit UX            | Plain-English nontechnical explanation of claim decisions, source chains, conflict history, and report integrity.        | Keep raw IDs available but do not force users to understand internals.                              |

## Appendix A - Important file paths

| Concern                   | Files / directories                                                                                                                          |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| App factory and routers   | src/atlas/presentation/api/app.py; src/atlas/presentation/api/routers/*.py; src/atlas/presentation/api/schemas/*.py.                         |
| Configuration/security    | src/atlas/config.py; src/atlas/security.py; src/atlas/presentation/api/dependencies.py; src/atlas/presentation/api/middleware.py.            |
| Core domain               | src/atlas/domain/entities.py; enums.py; exceptions.py; services/conflict_detector.py; projection_builder.py; winner_policy.py; ingestion.py. |
| Use cases                 | src/atlas/application/use_cases/*.py; application/unit_of_work.py; application/services/*.py.                                                |
| DB infrastructure         | src/atlas/infrastructure/db/orm_models.py; session.py; unit_of_work.py; repositories/*.py.                                                   |
| Migrations                | alembic/versions/*.py, current head 049_fk_covering_indexes.py.                                                                              |
| Frontend routes           | apps/web/src/app/router.tsx; apps/web/src/routes/*.tsx.                                                                                      |
| Frontend API/client/types | apps/web/src/lib/api.ts; apps/web/src/types/api.ts; apps/web/src/features/*.api.ts.                                                          |
| Ops                       | README.md; ARCHITECTURE.md; TENANT_RLS.md; OPERATIONS.md; deploy/free/*; ops/runbooks/*; ops/load/*.js.                                      |
| Patch bundle              | atlas-conflict-candidates-patch.zip: patch/PATCH_NOTES.md plus backend/frontend file replacements.                                           |

# Appendix B - Glossary

| Term                       | Meaning                                                                                                                      |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Evidence chain             | The chain from source/document/raw snapshot to claim to conflict/projection/history. This is the defensible truth substrate. |
| Projection                 | Derived current best explanation from active evidence and conflict decisions. Useful, but not the truth itself.              |
| Claim                      | A field/value assertion from a source or curator override. Claims are the atomic evidence units.                             |
| Conflict                   | A first-class disagreement between active claims about a field. Must be resolved/reopened with rationale.                    |
| CuratorOverride            | Seed source used for MANUAL_OVERRIDE claims. Do not edit projections directly.                                               |
| BFF                        | Backend-for-Frontend service that owns browser sessions and holds upstream API keys server-side.                             |
| RLS                        | PostgreSQL Row Level Security enforcing tenant payload isolation with app.current_tenant_id.                                 |
| HFACS/SHELO                | Aviation causality frameworks. Persisted public causal assertions must be evidence-backed and reviewed.                      |
| Hermes/Orion/Chronos/Argus | Source fetching, entity extraction, timeline reconstruction, and signal detection subsystems.                                |

# Appendix C - Handoff checklist for a coding AI

- Confirm whether you are working against the base backend/frontend bundle or the conflict-candidates patched state.
- Run or attempt: python -m compileall -q src tests alembic; ruff check .; mypy src; pytest --no-cov -m "not integration"; for frontend: npm ci && npm run build && npm run lint.
- For persistence changes, inspect alembic/versions and orm\_models.py before writing a migration.
- For API changes, inspect current routers/schemas and tests/api before adding endpoints.
- For UI changes, inspect src/lib/api.ts and existing route/component patterns. Keep apps/web talking to BFF only.
- For tenant/private changes, inspect TENANT\_RLS.md, create\_tenant\_uow, tenancy use cases, and public tenant-boundary tests.
- Document unavailable tooling and environment limits instead of implying checks passed.